

Τεχνολογίες Αποκεντρωμένων δεδομένων

1ο Πρότζεκτ

Πανεπιστήμιο Πατρών

Τμήμα Μηχανικών Η/Υ και Πληροφορικής

Ακαδημαϊκό Έτος 2023-2024

**Προσομοίωση Συστήματος Chord σε γλώσσα
προγραμματισμού Python**

ΜΕΛΗ ΟΜΑΔΑΣ

1ο μέλος

Ονοματεπώνυμο: ΓΚΟΥΡΓΚΟΥΤΑΣ ΒΑΣΙΛΕΙΟΣ

ΑΜ: 1084667

ΕΞΑΜΗΝΟ: 7

Email: up1084667@ac.upatras.gr

2ο μέλος

Ονοματεπώνυμο: ΚΟΝΤΟΓΙΩΡΓΟΣ ΑΝΑΣΤΑΣΙΟΣ

ΑΜ: 1090084

ΕΞΑΜΗΝΟ: 7

Email: up1090084@ac.upatras.gr

Εισαγωγή

Στο συγκεκριμένο project μας ζητήθηκε να υλοποιήσουμε ένα αποκεντρωμένο ευρετήριο βασισμένο σε κατανεμημένο κατακερματισμό στην γλώσσα προγραμματισμού Python. Το ευρετήριο θα έχει τη δυνατότητα να αποθηκεύει ένα σύνολο δεδομένων απο κείμενα που αφορούν επιστήμονες υπολογιστών και μέσω ενός web crawler θα αντλούμε τα στοιχεία από ένα συγκεκριμένο url. Στη συνέχεια μέσω του κώδικα που υλοποιήσαμε για το συγκεκριμένο ευρετήριο έχουμε τη δυνατότητα να κάνουμε τις βασικές πράξεις που μας ζητούνται όπως: lookup, node join και node leave.

Παραδοχές:

- 1) Αν εισάγουμε έναν κόμβο στο σύστημα και ο διάδοχος του κόμβος έχει περισσότερα από ένα δεδομένα, τότε ένα από τα δεδομένα με το κλειδί του θα μεταφερθούν σε εκείνον τον κόμβο που μόλις εισήχθει στο σύστημα.
- 2) Η υλοποίηση του δακτυλίου μας είναι τέτοια όπου ανατίθεται ένα κλειδί σε ένα κόμβο. Δεν υπάρχουν δηλαδή περισσότερα κλειδιά από ότι κόμβους.

Web Crawler

Η υλοποίηση που πραγματοποιήσαμε αφορά το σύστημα chord. Αρχικά αφού κατανοήσαμε τις βασικές πράξεις που χρειάζονται αλλά και το πως θα μπορέσουμε να αντλήσουμε και να πάρουμε τα δεδομένα μέσω του συγκεκριμένου url (https://en.wikipedia.org/wiki/List_of_computer_scientists), προχωρήσαμε δημιουργώντας αρχικά τον web crawler όπου αφορά την άντληση των δεδομένων από το παραπάνω url και κάνοντας <<crawling>> αντλούμε τα δεδομένα απο links μέσα σε αυτό το url.

Ο κώδικας για το web crawler είναι ο εξής:

```
import json
import requests
from bs4 import BeautifulSoup
from bs4.element import NavigableString
import re
import csv
```

```

links = [] # Περιέχει τους συνδέσμους του κάθε computer scientist στην
wikipedia
awards = [] # Περιέχει τα βραβεία του κάθε computer scientist στην
wikipedia
surnames1 = [] # Περιέχει τα επίθετα του κάθε computer scientist στην
wikipedia
institutions = [] # Περιέχει τα ιδρύματα σπουδών του κάθε computer
scientist στην wikipedia

# Ορισμός συνάρτησης με την οποία θα μπορέσουμε να αντλήσουμε το
επίθετο του κάθε επιστήμονα
# και παράλληλα και τα Links που αντιστοιχούν στο σύνδεσμο του κάθε
επιστήμονα
def get_computer_scientist_surnames(url, limit=682):
    response = requests.get(url)
    if response.status_code == 200:
        soup = BeautifulSoup(response.text, 'html.parser')
        ul_elements = soup.select('div.mw-parser-output ul')
        wiki = 'https://en.wikipedia.org/'
        surnames = []
        for ul in ul_elements:
            li_elements = ul.find_all('li')
            for li in li_elements:
                a_elements = li.find('a')
                link1 = a_elements.get("href")
                link = wiki+link1
                links.append(link)
                surname = a_elements.text.split()[-1]
                surnames.append(surname)
                if len(surnames) == limit:
                    return surnames

        return surnames
    else:
        print(f"Failed to retrieve the page. Status code:
{response.status_code}")
        return None

# Συνάρτηση όπου αντλούμε τα βραβεία του κάθε επιστήμονα με κατάλληλη
find και select
# με τη χρήση της βιβλιοθήκης BeautifulSoup
def crawl(url, awards):
    response = requests.get(url)

```

```

if response.status_code == 200:
    soup = BeautifulSoup(response.text, 'html.parser')
    awards_elements = soup.select('div.plainlist ul')
    awards_section = soup.find('th', {'scope': 'row', 'class':
'infobox-label'}, string='Awards')

    counter = 0
    if awards_section:
        # Πλοήγηση στο αντίστοιχο ul που περιέχει τα βραβεία
        ul = awards_section.find_next('ul')

        for li in ul.find_all('li'):
            a_element = li.find('a')
            if a_element:

                counter += 1

    awards.append(counter)

else:
    print(f"Failed to retrieve the page. Status code:
{response.status_code}")
    return None

# Συνάρτηση όπου αντιλούμε τα ιδρύματα του κάθε επιστήμονα με κατάλληλη
find και select
# με τη χρήση της βιβλιοθήκης BeautifulSoup
def crawl_institutions(url,institutions):
    response = requests.get(url)
    if response.status_code == 200:
        soup = BeautifulSoup(response.text, 'html.parser')

        institution_section = soup.find('th', {'scope': 'row', 'class':
'infobox-label'}, string='Institutions')

        if institution_section:
            # Πλοήγηση στο αντίστοιχο ul που περιέχει τα ιδρύματα

```

```

        td = institution_section.find_next('td')

        for a in td.find_next('a'):
            if a:
                institutions.append(a.get_text())
            else:
                institutions.append("")

        else:
            print(f"Failed to retrieve the page. Status code:
{response.status_code}")
            return None

url_to_crawl =
'https://en.wikipedia.org/wiki/List_of_computer_scientists'
surnames = get_computer_scientist_surnames(url_to_crawl, limit=683)

if surnames:
    surnames1 = surnames.copy()
    print(surnames1)
    print(len(surnames1))
else:
    print("No surnames were extracted.")

# Με τη χρήση των δύο for παίρνουμε τα δεδομένα που χρειαζόμαστε δηλαδή
τα βραβεία και τα ιδρύματα
for i in range(0,682):
    crawl(links[i],awards)

for i in range(0,682):
    crawl_institutions(links[i],institutions)

print(institutions)
print(len(institutions))
csv_file = 'output3.csv'

#Συνδυασμός λιστών σε μια λίστα πλειάδων
data = list(zip(surnames1 , awards, institutions))

# Άνοιγμα csv file με τη μέθοδο write
with open(csv_file, 'w', encoding='utf-8' ,newline='') as file:
    # Δημιουργία ενός αντικειμένου εγγραφής CSV

```

```
writer = csv.writer(file)
# Εγγραφή των δεδομένων στο csv file
writer.writerows(data)

print(f'Data has been written to {csv_file}')
```

Αρχικά στον κωδικά μας κάνουμε import ορισμένες βιβλιοθήκες που θα μας βοηθήσουν στην εκτέλεση του προγράμματος. Στη συνέχεια δημιουργούνται συγκεκριμένες λίστες (links=[], awards=[], surnames1=[], institutions=[]), όπου η κάθε λίστα ξεχωριστά θα περιέχει τα links απο τους συνδέσμους των κάθε computer scientists, τα βραβεία που έχουν πάρει οι επιστήμονες, τα επίθετα τους καθώς και τα ιδρύματα απο τα οποία έχουν αποφοιτήσει. Έπειτα με τη δημιουργία της συνάρτησης **get_computer_scientist_surnames()** και δίνοντας σαν όρισμα το url το οποίο μας δόθηκε, αντλούμε τα επίθετα των επιστημόνων και παράλληλα και τα link του κάθε συνδέσμου που ανήκουν στον εκάστοτε επιστήμονα. Στη συνέχεια, με τη συνάρτηση crawl και ορίζοντας κατάλληλη find και select με την χρήση της βιβλιοθήκης BeautifulSoup εντοπίζουμε τα βραβεία του κάθε επιστήμονα. Η ίδια διαδικασία συμβαίνει και με τη συνάρτηση **crawl_institutions** που σε αυτή τη περίπτωση με τον ορισμό κατάλληλης find και select εντοπίζουμε το ίδρυμα του κάθε επιστήμονα και έπειτα με δύο for καλώντας τις συναρτήσεις που υλοποιήσαμε, παίρνουμε τα δεδομένα που χρειαζόμαστε δηλαδή τα βραβεία και τα ιδρύματα. Τέλος δημιουργούμε ένα csv αρχείο στο οποίο εισάγουμε τα δεδομένα που μόλις πήραμε από τις λίστες μας.

CHORD

Μετά τη δημιουργία του προγράμματος crawler, προχωρήσαμε στην υλοποίηση του κώδικα chord. Όπως προαναφέραμε και στην αρχή το πρόγραμμα chord που υλοποιήθηκε αφορά ένα αποκεντρωμένο ευρετήριο βασισμένο σε κατανεμημένο κατακερματισμό, όπου μπορεί να λαμβάνει ένα σύνολο δεδομένων και να πραγματοποιεί κάποιες βασικές διαδικασίες ή πιο συγκεκριμένα κάποιες βασικές πράξεις όπως είναι το: lookup, node join και node leave. Τα δεδομένα που τοποθετήθηκαν στους κόμβους αντλήθηκαν από τον crawler και το αρχείο csv από όπου προήλθαν.

Ξεκινήσαμε την υλοποίηση του προγράμματος, δημιουργώντας αρχικά την κλάση `ChordRing` η οποία υλοποιεί το δακτύλιο που θα εμπεριέχει τους κόμβους του συστήματος `Chord`. Θέτουμε την συνάρτηση `__init__` με ορίσματα τριών μεταβλητών (`self`, `nodes`, `ids`) και αρχικοποιούμε την λίστα των κόμβων στο δακτύλιο του συστήματος, τη λίστα με τα IDs των κόμβων του συστήματος, το id του πρώτου κόμβου και το id του τελευταίου κόμβου.

Επίσης ορίζουμε τη συνάρτηση `update`, ώστε να ενημερώνουμε το δακτύλιο κάθε φορά όταν γίνεται εισαγωγή ή διαγραφή κάποιου κόμβου και ορίζουμε μέσα στη συνάρτηση `update` το πρώτο κόμβο του δακτυλίου σε μία μεταβλητή `start_node`, σε μία μεταβλητή `end_node` ορίζουμε το τελευταίο κόμβο του δακτυλίου, τοποθετούμε τα id που αποθηκεύουμε στην λίστα με τα `ids` σε δύο μεταβλητές και κάνουμε `sorting` την λίστα. Στη συνέχεια υλοποιήσαμε την κλάση `ChordNode`, κλάση η οποία αφορά τον κάθε κόμβο και ανάλογα με την ενέργεια που θέλουμε να ακολουθήσουμε, αξιοποιούμε τις συναρτήσεις που έχουμε δημιουργήσει μέσα στη συγκεκριμένη κλάση. Τα ορίσματα που έχει η κλάση και έχουν αρχικοποιηθεί είναι το id του κόμβου, ο διάδοχος κόμβος, ο προκάτοχος κόμβος, ένας πίνακας που αποθηκεύει τα `finger`, ένας δακτύλιος στον οποίο ανήκει ο κόμβος και μία μεταβλητή που αποθηκεύονται τα κλειδιά μαζί με τα δεδομένα που θα τοποθετηθούν στους κόμβους. Μέσα στην κλάση υλοποιήθηκε μία συνάρτηση `add_key_value_pair()`, όπου κάνει προσθήκη ζεύγους κλειδίου-τιμής στο κόμβο όπου το κλειδί θα είναι το πεδίο `education` και η τιμή θα έχει τα πεδία `surname` και `awards`. Μία ακόμα συνάρτηση είναι η `finger_table()` που υπολογίζει το `finger` για το κάθε κόμβο και με κλήση της `find_successor` συνάρτησης εντοπίζει κάθε φορά τον σωστό κόμβο, ώστε να αποθηκευτεί στο `finger table`. Επίσης έχει δημιουργηθεί η συνάρτηση `update_successor_predecessor()`, όπου κάθε φορά ανανεώνει τον `predecessor` και τον `successor` και τους ενημερώνει αντίστοιχα για τον κάθε κόμβο. Στην συνέχεια υλοποιήσαμε και την συνάρτηση `find_successor_id()`, όπου εντοπίζουμε με την συγκεκριμένη συνάρτηση τον `successor` του κάθε κόμβου με βάση το κλειδί που δώσαμε σαν όρισμα και επιστρέφουμε ύστερα από συγκρίσεις με το κλειδί, τον κόμβο ως `successor`. Με παρόμοιο τρόπο λειτουργεί η `find_successor` αλλά για τα ίδια τα στιγμιότυπα της κλάσης `ChordNode`. Ακόμα οι συναρτήσεις που υλοποιήσαμε είναι η `join()`, όπου τοποθετεί ένα νέο κόμβο στο δακτύλιο, ενημερώνει τους `successor-predecessor` και προσθέτει στο νέο κόμβο ένα από τα δεδομένα του διαδόχου κόμβου. Μετέπειτα προχωρήσαμε στην υλοποίηση της συνάρτησης `leave()` που διαγράφει ένα κόμβο από τον δακτύλιο, ενημερώνει αντίστοιχα τους `successor-predecessor` και επιπλέον κάνει μεταφορά δεδομένων στον διάδοχο κόμβο που είχε πριν φύγει από τον δακτύλιο. Η

συνάρτηση **lookup()** ψάχνει το κλειδί ενός κόμβου και επιστρέφει το περιεχόμενο του. Έπειτα υλοποιήσαμε την συνάρτηση **create_chord_ring()** που δημιουργεί το δακτύλιο του chord και ενημερώνει τους κόμβους. Η συνάρτηση **print_successor_predecessor()** εκτυπώνει τους διαδόχους και τους προκατόχους και η **print_node_data()** εκτυπώνει τα δεδομένα του κάθε κόμβου. Επιπροσθέτως υπάρχει η συνάρτηση **hash_function()** που αφορά των κατακερματισμό των Ids. Τέλος δημιουργήσαμε το μενού με τις επιλογές που μπορεί να διαλέξει ο χρήστης με τις διάφορες λειτουργίες που επιθυμεί, καθώς και τη συνάρτηση **load_data_from_csv()** που φορτώνει τα αποτελέσματα που ανακτήθηκαν από τον crawler στον δακτύλιο και στους κόμβους. Αξιοποιώντας το menu στην main μπορούμε να εκτελέσουμε το σύστημα chord που υλοποιήσαμε. Αρχικά με την **πρώτη επιλογή** έχουμε τη δυνατότητα να εισάγουμε το id του νέου κόμβου και να τοποθετήσουμε έναν καινούριο κόμβο στον δακτύλιο. Με την **επιλογή νούμερο δύο** εισάγουμε το id του κόμβου που επιθυμούμε να διαγράψουμε. Εάν υπάρχουν δεδομένα στον κόμβο που πρόκειται να διαγραφεί τοποθετούνται στον διάδοχο κόμβο του όπως προαναφέραμε. Με αυτόν τον τρόπο εξασφαλίζουμε ότι τα δεδομένα δεν χάθηκαν με την διαγραφή του κόμβου. Στη συνέχεια με την **επιλογή νούμερο τρία** εμφανίζουμε τους successor-predecessor των κόμβων. Με την **επιλογή νούμερο τέσσερα** εμφανίζουμε τους κόμβους του δακτυλίου με τα δεδομένα τους και με την **επιλογή νούμερο πέντε** γίνεται η διαδικασία του lookup δίνοντας ως τιμή το id ενός κόμβου και επιστρέφοντας τον αντίστοιχο κόμβο με τα δεδομένα του. Επιπρόσθετα με την **επιλογή νούμερο έξι** του menu επιλογών, μπορούμε να αναζητήσουμε δεδομένα ενός κόμβου θέτοντας προϋποθέσεις όπως το πανεπιστήμιο που φοίτησε κάποιος επιστήμονας και ορίζουμε ένα κάτω όριο βραβείων ώστε όταν ο χρήστης εισάγει τον αριθμό τους να εμφανίζονται κατάλληλα τα δεδομένα. Δηλαδή εκτελούνται query της μορφής: **«Βρείτε τους επιστήμονες της επιστήμης υπολογιστών από τη ΒΔ Wikipedia που έκαναν βασικές σπουδές στο Harvard και έχουν αποσπάσει > 2 βραβεία»**. Τέλος με τη **επιλογή νούμερο επτά** κάνουμε έξοδο από το σύστημα chord. Στον κώδικα τόσο του αρχείου crawler_chord.py όσο και του Chord.py υπάρχουν αναλυτικά σχόλια που επεξηγούν τι κάνει η κάθε συνάρτηση και ο συνολικός κώδικας ώστε να υπάρχει διευκόλυνση στην κατανόηση.

Εκτέλεση του συστήματος Chord

Για την εκτέλεση του προγράμματος στην γλώσσα προγραμματισμού python αξιοποιούμε το μενού επιλογών που έχουμε δημιουργήσει.

Καλώς ήρθατε στον Δακτύλιο Chord:

1. Εισαγωγή Κόμβου
 2. Διαγραφή Κόμβου
 3. Εκτύπωση Διαδόχων και Προκατόχων
 4. Εκτύπωση Δεδομένων κάθε Κόμβου
 5. Αναζήτηση δεδομένων με βάση το κλειδί
 6. Αναζήτηση σε κόμβο με βάση το Πανεπιστήμιο και τον αριθμό των βραβείων
 7. Έξοδος
- Επιλέξτε την επιλογή σας (1-7):

Αρχικά αξιοποιώντας την τέταρτη επιλογή στο σύστημα Chord μπορούμε να εκτυπώσουμε τα δεδομένα κάθε κόμβου.

Επιλέξτε την επιλογή σας (1-7): 4

```
Node 0: {'Ajman University': {'surname': 'Khan', 'awards': '0'}}
Node 1: {'RWTH Aachen University': {'surname': 'Aalst', 'awards': '0'}}
Node 2: {'University of Texas at Austin': {'surname': 'Aaronson', 'awards': '4'}}
Node 3: {'University of California, Berkeley': {'surname': 'Abebe', 'awards': '0'}}
Node 4: {'Massachusetts Institute of Technology': {'surname': 'Abelson', 'awards': '1'}}
Node 5: {'INRIA': {'surname': 'Abiteboul', 'awards': '4'}}
Node 6: {'University of Oxford': {'surname': 'Abramsky', 'awards': '4'}}
Node 7: {'University of Southern California': {'surname': 'Adleman', 'awards': '2'}}
Node 8: {'Indian Institute of Technology Kanpur': {'surname': 'Agrawal', 'awards': '8'}}
Node 9: {'Carnegie Mellon University': {'surname': 'Ahn', 'awards': '3'}}
Node 10: {'Columbia University': {'surname': 'Aho', 'awards': '5'}}
Node 11: {'IBM': {'surname': 'Allen', 'awards': '6'}}
Node 12: {'Thesis': {'surname': 'Amdahl', 'awards': '4'}}
Node 13: {'University of California, Berkeley': {'surname': 'Anderson', 'awards': '5'}}
Node 14: {'': {'surname': 'Anthony', 'awards': '0'}}
Node 15: {'': {'surname': 'Appel', 'awards': '0'}}
Node 16: {'University of Washington': {'surname': 'Aragon', 'awards': '4'}}
Node 17: {'': {'surname': 'Arden', 'awards': '0'}}
Node 18: {'': {'surname': 'Jones', 'awards': '0'}}
Node 19: {'Princeton University': {'surname': 'Arora', 'awards': '0'}}
Node 20: {'': {'surname': 'Asprey', 'awards': '0'}}
Node 21: {'': {'surname': 'Atanasoff', 'awards': '17'}}
Node 22: {'': {'surname': 'Atre', 'awards': '0'}}
Node 23: {'Trinity College, Cambridge': {'surname': 'Babbage', 'awards': '2'}}
Node 24: {'Dow Chemical': {'surname': 'Bachman', 'awards': '5'}}
Node 25: {'Royal Aircraft Establishment': {'surname': 'Backhouse', 'awards': '0'}}
Node 26: {'IBM': {'surname': 'Backus', 'awards': '9'}}
Node 27: {'IBM Watson Research Center': {'surname': 'Bacon', 'awards': '3'}}
Node 28: {'New Jersey Institute of Technology': {'surname': 'Bader', 'awards': '1'}}
```

Επιπλέον με την τρίτη επιλογή στο μενού, μπορούμε να δούμε τον προκάτοχο και τον διάδοχο κάθε κόμβου.

```
Επιλέξτε την επιλογή σας (1-7): 3
Node 0: Successor - 1, Predecessor - 681
Node 1: Successor - 2, Predecessor - 0
Node 2: Successor - 3, Predecessor - 1
Node 3: Successor - 4, Predecessor - 2
Node 4: Successor - 5, Predecessor - 3
Node 5: Successor - 6, Predecessor - 4
Node 6: Successor - 7, Predecessor - 5
Node 7: Successor - 8, Predecessor - 6
Node 8: Successor - 9, Predecessor - 7
Node 9: Successor - 10, Predecessor - 8
Node 10: Successor - 11, Predecessor - 9
Node 11: Successor - 12, Predecessor - 10
Node 12: Successor - 13, Predecessor - 11
Node 13: Successor - 14, Predecessor - 12
Node 14: Successor - 15, Predecessor - 13
Node 15: Successor - 16, Predecessor - 14
Node 16: Successor - 17, Predecessor - 15
Node 17: Successor - 18, Predecessor - 16
Node 18: Successor - 19, Predecessor - 17
Node 19: Successor - 20, Predecessor - 18
Node 20: Successor - 21, Predecessor - 19
Node 21: Successor - 22, Predecessor - 20
Node 22: Successor - 23, Predecessor - 21
Node 23: Successor - 24, Predecessor - 22
Node 24: Successor - 25, Predecessor - 23
Node 25: Successor - 26, Predecessor - 24
Node 26: Successor - 27, Predecessor - 25
Node 27: Successor - 28, Predecessor - 26
```

Έστω ότι θέλουμε να διαγράψουμε κάποιο κόμβο από το σύστημα Chord ή ότι κάποιος κόμβος έχει πέσει και δεν είναι διαθέσιμος. Τότε με την δεύτερη επιλογή μπορούμε να απομακρύνουμε τον κόμβο από το σύστημα και τα δεδομένα του θα τοποθετηθούν σε κάποιον άλλο κόμβο και πιο συγκεκριμένα στον διάδοχο κόμβο που είχε πριν εγκαταλείψει το σύστημα του δακτυλίου. Παρακάτω δίνεται ένα παράδειγμα από την διαγραφή ενός κόμβου.

Διαγραφή Κόμβου

Έστω ότι διαγράφουμε τον κόμβο 4.

```
Node 3: {'University of California, Berkeley': {'surname': 'Abebe', 'awards': '0'}}
Node 4: {'Massachusetts Institute of Technology': {'surname': 'Abelson', 'awards': '1'}}
Node 5: {'INRIA': {'surname': 'Abiteboul', 'awards': '4'}}
```

Επιλέξτε την επιλογή σας (1-7): 2
Εισαγωγή του ID του κόμβου που θέλετε να διαγράψετε: 4

Εάν εκτελέσουμε την τέταρτη επιλογή θα παρατηρήσουμε ότι ο κόμβος 4 έχει φύγει από το σύστημα.

```
Επιλέξτε την επιλογή σας (1-7): 4
Node 0: {'Ajman University': {'surname': 'Abelson', 'awards': '1'}}
Node 1: {'RWTH Aachen University': {'surname': 'Abelson', 'awards': '1'}}
Node 2: {'University of Texas at Austin': {'surname': 'Abelson', 'awards': '1'}}
Node 3: {'University of California, Berkeley': {'surname': 'Abebe', 'awards': '0'}}
Node 5: {'INRIA': {'surname': 'Abiteboul', 'awards': '4'}}
```

Και τα δεδομένα του έχουν τοποθετηθεί στον κόμβο 5 δηλαδή στον διάδοχό του.

```
Node 3: {'University of California, Berkeley': {'surname': 'Abebe', 'awards': '0'}}
Node 5: {'INRIA': {'surname': 'Abiteboul', 'awards': '4'}, 'Massachusetts Institute of Technology': {'surname': 'Abelson', 'awards': '1'}}
```

Εισαγωγή Κόμβου

Εάν τώρα θέλουμε να εισάγουμε ένα κόμβο στο σύστημα μας, θα εκτελέσουμε την πρώτη επιλογή από το μενού και τα δεδομένα από τον κόμβο ο οποίος θα γίνει ο διάδοχός του και που είναι περισσότερα από ένα (τα δεδομένα) θα τοποθετηθούν στο καινούριο κόμβο που εισήχθει και θα φύγουν από τον διάδοχο κόμβο.

Έστω ότι ο δακτύλιος μας έχει αυτήν την μορφή:

```
Node 678: {'': {'surname': 'Zaman', 'awards': '0'}}
Node 680: {'Columbia University': {'surname': 'Zedan', 'awards': '0'}, 'Carnegie Mellon University': {'surname': 'Zdonik', 'awards': '0'}}
```

Και έστω ότι εισάγουμε τον κόμβο 679.

Επιλέξτε την επιλογή σας (1-7): 1
Εισαγωγή του ID του νέου κόμβου: 679

Τότε ένα από τα δεδομένα του διαδόχου κόμβου θα μεταφερθεί μαζί με το κλειδί στον κόμβο που μόλις εισήχθει. Δηλαδή θα μεταφερθεί το δεδομένο από τον κόμβο 680 στον κόμβο 679.

```
Node 678: {'': {'surname': 'Zaman', 'awards': '0'}}
Node 679: {'Columbia University': {'surname': 'Zedan', 'awards': '0'}}
Node 680: {'Carnegie Mellon University': {'surname': 'Zdonik', 'awards': '0'}}
```

Επίσης με την επιλογή τρία εκτυπώνοντας τους successors και predecessors παρατηρούμε ότι ο κόμβος 679 τοποθετήθηκε σωστά στο σύστημα μας και έλαβε τον κατάλληλο προκάτοχο και διάδοχο.

Επιλέξτε την επιλογή σας (1-7): 3

```
Node 678: Successor - 679, Predecessor - 677
Node 679: Successor - 680, Predecessor - 678
Node 680: Successor - 681, Predecessor - 679
```

Αναζήτηση lookup

Με την πέμπτη επιλογή μπορούμε να αναζητήσουμε κάποιον κόμβο στον δακτύλιο Chord. Έστω ότι θέλουμε να αναζητήσουμε τον κόμβο 670. Τότε δίνοντας σαν όρισμα στο τερματικό το κλειδί του κόμβου, εμφανίζεται ο κόμβος με τα δεδομένα που περιέχει εκείνη την στιγμή.

```
Επιλέξτε την επιλογή σας (1-7): 5
Εισαγωγή του κλειδιού για αναζήτηση: 670
Ο κόμβος που αντιστοιχεί στο κλειδί 670 είναι ο κόμβος 670
Δεδομένα του κόμβου 670: {'ETH Zurich': {'surname': 'Yannakakis', 'awards': '1'}}
```

Αναζήτηση με βάση το πανεπιστήμιο και τον αριθμό των βραβείων

Τέλος εάν θέλουμε να αναζητήσουμε κάποιον επιστήμονα με βάση το πανεπιστήμιο όπου σπούδασε και εισάγοντας ένα κάτω όριο όσον αφορά τον αριθμό βραβείων, εκτελούμε την έκτη επιλογή. Έστω ότι θέλουμε να αναζητήσουμε σε ένα κόμβο (για παράδειγμα στον 681) τους επιστήμονες που φοίτησαν στο Stanford University και έχουν αριθμό βραβείων μεγαλύτερο από

0. Τότε λαμβάνουμε ως αποτέλεσμα τα δεδομένα που πληρούν αυτά τα κριτήρια.

```
Επιλέξτε την επιλογή σας (1-7): 6
Εισαγωγή του ID του κόμβου για την αναζήτηση: 681
Εισαγωγή του Πανεπιστημίου: Stanford University
Εισαγωγή του αριθμού των Βραβείων: 0
Οι επιστήμονες που έκαναν βασικές σπουδές στο Stanford University και έχουν αποσπάσει > 0 βραβεία είναι: Zilberstein
```

Εάν κάποιος κόμβος δεν πληρεί τα ζητούμενα ή δεν έχει αποθηκευμένα τα δεδομένα που αναζητούμε, τότε εμφανίζεται κατάλληλο μήνυμα.

Κόμβος που δεν περιέχει το δεδομένο:

```
Επιλέξτε την επιλογή σας (1-7): 6
Εισαγωγή του ID του κόμβου για την αναζήτηση: 678
Εισαγωγή του Πανεπιστημίου: Stanford University
Εισαγωγή του αριθμού των Βραβείων: 0
Δεν βρέθηκαν επιστήμονες που πληρούν τα κριτήρια.
```

Κόμβος που δεν υπάρχει στο σύστημα:

```
Επιλέξτε την επιλογή σας (1-7): 6
Εισαγωγή του ID του κόμβου για την αναζήτηση: 1006
Εισαγωγή του Πανεπιστημίου: Stanford University
Εισαγωγή του αριθμού των Βραβείων: 0
Δεν βρέθηκαν επιστήμονες που πληρούν τα κριτήρια.
```

Καταληκτικά με την έβδομη επιλογή κάνουμε έξοδο από το σύστημα Chord.

```
Επιλέξτε την επιλογή σας (1-7): 7
Έξοδος από το πρόγραμμα. Αντίο!
```

Ο κώδικας για το σύστημα Chord είναι ο εξής:

```
import csv

numberOfData = 682 # Πλήθος δεδομένων που θα εισαχθούν στο δακτύλιο
```

```

class ChordRing:
    def __init__(self, nodes, ids):
        self.Nodes_List = nodes # Λίστα κόμβων στο δακτύλιο του
        συστήματος Chord
        self.Nodes_Ids = ids # Λίστα με τα IDs των κόμβων του
        συστήματος Chord
        self.start_node_id = 0 # Εδώ αρχικοποιούμε το ID του πρώτου
        κόμβου
        self.end_node_id = 0 # Εδώ αρχικοποιούμε το ID του
        τελευταίου κόμβου

    def update(self):
        # Καλούμε κάθε φορά την συνάρτηση για να ενημερώσουμε τον
        δακτύλιο
        self.start_node = self.Nodes_List[0] # Τοποθετούμε τον πρώτο
        κόμβο του δακτυλίου σε μία μεταβλητή start_node
        self.end_node = self.Nodes_List[len(self.Nodes_List) - 1] #
        Όμοια για το τελευταίο στοιχείο του δακτυλίου
        self.start_node_id = self.Nodes_Ids[0] # Τοποθετούμε τα id
        που αποθηκεύουμε στην λίστα με τα ids σε δύο μεταβλητές
        self.end_node_id = self.Nodes_Ids[len(self.Nodes_Ids) - 1]
        self.Nodes_List.sort(key=lambda n: n.node_id) # Κάνουμε
        sorting την λίστα με τους κόμβους που έχουμε στον δακτύλιο κάθε φορά
        για να τοποθετούνται οι κόμβοι στην σωστή σειρά

# Κλάση η οποία αφορά τον κάθε κόμβο και ανάλογα με την ενέργεια που
# θέλουμε να ακολουθήσουμε,
# αξιοποιούμε τις συναρτήσεις που έχουμε υλοποιήσει στην κλάση αυτή
class ChordNode:
    def __init__(self, node_id, chord_ring): # Ο constructor της
        κλάσης μας
        self.node_id = node_id # ID του κόμβου
        self.successor = None # Διάδοχος κόμβος
        self.predecessor = None # Προκάτοχος κόμβος
        self.finger = [] # Πίνακας που θα αποθηκεύει τα finger
        self.ring = chord_ring # Δακτύλιος στον οποίο ανήκει ο
        κόμβος και σχετίζεται με την κλάση ChordRing
        self.key_store = {} # Εδώ αποθηκεύονται τα κλειδιά μαζί με τα
        δεδομένα που θα τοποθετηθούν στους κόμβους

    def add_key_value_pair(self, key, surname, awards):
        # Προσθήκη ζεύγους κλειδιού-τιμής στον κόμβο όπου το κλειδί

```



```

θα είναι το πεδίο education και
    # και η τιμή θα έχει τα πεδία surname και awards
    self.key_store[key] = {'surname': surname, 'awards': awards}

def finger_table(self):
    # Υπολογισμός του finger table για κάθε κόμβο
    for i in range(0, m):
        s = (self.node_id + (2**i)) % (2**m)
        self.finger.append(self.find_successor_id(s)) # Κλήση της
find_successor ώστε να εντοπίζουμε κάθε φορά τον σωστό κόμβο, ώστε να
αποθηκευτεί στο finger table

    def update_successor_predecessor(self): # Συνάρτηση η οποία
ανανεώνει κάθε φορά τον predecessor και τον successor κάθε κόμβου
        counter = 0

        # Ενημέρωση του successor και predecessor του κόμβου
        if self.node_id == self.ring.start_node_id: # Εάν ο τρέχοντας
κόμβος ισούται με τον αρχικό κόμβο στον δακτύλιο
            self.successor = self.ring.Nodes_List[1]
            self.predecessor =
self.ring.Nodes_List[len(self.ring.Nodes_List) - 1]
        elif self.node_id == self.ring.end_node_id: # Εάν ο τρέχοντας
κόμβος είναι ο τελευταίος, τότε ανέθεσε σαν successor τον πρώτο κόμβο
            self.successor = self.ring.start_node
            self.predecessor =
self.ring.Nodes_List[len(self.ring.Nodes_List) - 2]
        else: # Διαφορετικά αναζήτησε εσωτερικά στον δακτύλιο
            # Προσαρμόζουμε για κάθε κόμβο τον successor και τον
predecessor του
            for i in range(0, len(self.ring.Nodes_List)):
                if self.ring.Nodes_List[i].node_id < self.node_id:
                    counter += 1

            self.successor = self.ring.Nodes_List[(counter + 1) %
len(self.ring.Nodes_List)]
            self.predecessor = self.ring.Nodes_List[(counter - 1) %
len(self.ring.Nodes_List)]

    def find_successor_id(self, key): # Με την συγκεκριμένη συνάρτηση
εντοπίζουμε τον successor κάθε κόμβου με βάση το κλειδί που δώσαμε
σαν όρισμα

```



```

        if self.node_id == key: # Εάν το id του κόμβου ισούται με το
κλειδί, τότε επέστρεψε σαν successor τον ίδιο τον κόμβο
            return self.node_id
        elif self.node_id < key <= self.successor.node_id: # Εάν το
κλειδί είναι μεγαλύτερο απο τον τρέχοντα κόμβο και μικρότερο ή ίσο
από τον successor εκείνου του κόμβου, επέστρεψε τον successor του
κόμβου όπου αναζητούμε αυτή την στιγμή
            return self.successor.node_id
        elif self.successor.node_id < self.node_id and (key <
self.successor.node_id or self.node_id < key): # Εάν ο κόμβος είναι ο
τελευταίος στον δακτύλιο, τότε επέστρεψε τον successor του τελευταίου
κόμβου
            return self.successor.node_id
        else: # Εάν δεν ισχύουν τα παραπάνω, τότε καλούμε αναδρομικά
την συνάρτηση μεχρι να εντοπίσουμε τον successor που αναζητούμε με
βάση το κλειδί που δώσαμε σαν όρισμα
            return self.successor.find_successor_id(key)

def find_successor(self, key): # Λειτουργεί με παρόμοιο τρόπο
όπως η find_successor_id και αξιοποιείται στην συνάρτηση lookup για
την εύρεση κάποιου κόμβου με βάση το κλειδί

        if self.node_id == key:
            return self
        elif self.node_id < key <= self.successor.node_id:
            return self.successor
        elif self.successor.node_id < self.node_id and (key <
self.successor.node_id or self.node_id < key):
            return self.successor
        else:
            return self.successor.find_successor(key)

def join(self, chord_ring):
    # Τοποθέτηση του κόμβου στο δακτύλιο
    self.ring = chord_ring
    self.ring.Nodes_List.append(self) # Προσθήκη του κόμβου στη
λίστα των κόμβων
    self.ring.Nodes_Ids.append(self) # Προσθήκη του κόμβου στη
λίστα με τα Ids των κόμβων
    self.ring.update() # Ενημέρωση του δακτυλίου με τη νέα

```

προσθήκη

```
# Ενημέρωση των successor και predecessor
self.update_successor_predecessor() # Ενημέρωση του
διαδόχου και προκατόχου του νέου κόμβου
for nodes in self.ring.Nodes_List:
    nodes.update_successor_predecessor() # Ενημέρωση των
διαδόχων και προκατόχων όλων των κόμβων
self.finger_table() # Ενημέρωση του πίνακα finger του κόμβου

# Εύρεση του διαδόχου του νέου κόμβου που κάναμε εισαγωγή
successor_index = (self.ring.Nodes_List.index(self) + 1) %
len(self.ring.Nodes_List)
successor = self.ring.Nodes_List[successor_index]

# Μεταφορά ενός στοιχείου από τον διάδοχο κόμβο, στον νέο
κόμβο μόλις εισήχθει
if successor != self:
    # Παίρνουμε το κλειδί και το ένα από τα δεδομένα του
διαδόχου κόμβου
    keys_to_transfer = list(successor.key_store.keys())
    if keys_to_transfer:
        # Επιλογή του κλειδιού για μεταφορά απο την λίστα
        key_to_transfer = keys_to_transfer[0]
        # Αφαίρεση του κλειδιού και ενός από τα δεδομένα από
τον διάδοχο κόμβο ώστε να τοποθετηθούν στον καινούριο κόμβο
        value_to_transfer =
successor.key_store.pop(key_to_transfer)
        # Προσθήκη του κλειδιού και ενός από τα δεδομένα στον
νέο κόμβο
        self.key_store[key_to_transfer] = value_to_transfer

def leave(self):
    # Αποχώρηση του κόμβου από τον δακτύλιο
    if self.ring is not None:
        if self in self.ring.Nodes_Ids:
            self.ring.Nodes_Ids.remove(self)
            self.ring.Nodes_List.remove(self)
            self.ring.update()

    # Εύρεση νέου διαδόχου χρησιμοποιώντας την find_successor
```

```

        new_successor_id = self.find_successor_id(self.node_id +
1)

        new_successor =
self.ring.Nodes_List[0].find_successor(new_successor_id)

        # Μεταφορά δεδομένων στο νέο διάδοχο
        new_successor.key_store.update(self.key_store)

        # Αφαίρεση των δεδομένων από τον αποχωρούντα κόμβο
        self.key_store = {}

        # Ενημέρωση των κόμβων για τους νέους διαδόχους
        for nodes in self.ring.Nodes_List:
            nodes.update_successor_predecessor()
        self.finger_table()

def lookup(self, key):
    # Συνάρτηση για την αναζήτηση του κόμβου που αντιστοιχεί στο
κλειδί
    successor_id = self.find_successor_id(key)

    # Βρέθηκε ο κόμβος που αντιστοιχεί στο κλειδί
    if successor_id == self.node_id:
        return self
    else:
        # Καλεί τη συνάρτηση find_successor για τον πραγματικό
κόμβο
        return self.find_successor(key)

def create_chord_ring(m, ids):
    # Δημιουργία του δακτυλίου Chord και ενημέρωση των κόμβων
    chord_create = [ChordNode(i, None) for i in ids]
    chord_ring = ChordRing(chord_create, ids)
    for node in chord_create:
        node.ring = chord_ring
        node.update_successor_predecessor()

    return chord_ring

```

```

def print_successor_predecessor(chord_ring):
    # Εκτύπωση των διαδόχων και προκατόχων κάθε κόμβου (predecessors
και successors)
    for node in chord_ring.Nodes_List:
        print(f"Node {node.node_id}: Successor -
{node.successor.node_id}, Predecessor - {node.predecessor.node_id}")

def print_node_data(chord_ring):
    # Εκτύπωση των δεδομένων κάθε κόμβου
    for node in chord_ring.Nodes_List:
        print(f"Node {node.node_id}: {node.key_store}")

def display_menu():
    # Εμφάνιση μενού επιλογών
    print("Καλώς ήρθατε στον Δακτύλιο Chord:")
    print("1. Εισαγωγή Κόμβου")
    print("2. Διαγραφή Κόμβου")
    print("3. Εκτύπωση Διαδόχων και Προκατόχων")
    print("4. Εκτύπωση Δεδομένων κάθε Κόμβου")
    print("5. Αναζήτηση δεδομένων με βάση το κλειδί")
    print("6. Αναζήτηση σε κόμβο με βάση το Πανεπιστήμιο και τον
αριθμό των βραβείων")
    print("7. Έξοδος")

def load_data_from_csv(filename):
    # Φόρτωση των δεδομένων που ανακτήθηκαν από τον crawler και
τοποθετήθηκαν στο CSV αρχείο
    data = []

    with open(filename, 'r', encoding='UTF-8') as file:
        reader = csv.reader(file)
        for row in reader:
            surname, awards, education = row
            data.append({'surname': surname, 'awards': awards,
'education': education})

    return data

def hash_function(value, m):
    # Υποθέτουμε ότι η hash function θα επιστρέφει την τιμή μεταξύ 0
και 2^m-1
    return hash(value) % (2**m)

```

```

if __name__ == "__main__":
    # Φόρτωση των δεδομένων από το CSV αρχείο
    csv_data = load_data_from_csv('output3.csv')
    m = 10
    ids = [hash_function(i, m) for i in range(numberofData)]

    # Δημιουργία του δακτυλίου Chord και εισαγωγή των δεδομένων στους
    # κόμβους
    chord_ring = create_chord_ring(m, ids)

    for i, node in enumerate(chord_ring.Nodes_List):
        if i < len(csv_data):
            entry = csv_data[i]
            key = entry['education']
            surname = entry['surname']
            awards = entry['awards']
            node.add_key_value_pair(key, surname, awards)

    # Είσοδος στο κύριο μενού
    while True:
        display_menu()
        choice = input("Επιλέξτε την επιλογή σας (1-7): ")
        # Εισαγωγή νέου κόμβου στον δακτύλιο
        if choice == "1":
            put_id = int(input("Εισαγωγή του ID του νέου κόμβου: "))
            n = ChordNode(put_id, None)
            n.join(chord_ring)

            # Διαγραφή κάποιου κόμβου από τον δακτύλιο
            elif choice == "2":
                put_id = int(input("Εισαγωγή του ID του κόμβου που θέλετε
να διαγράψετε: "))
                for nodes in chord_ring.Nodes_List:
                    if nodes.node_id == put_id:
                        nodes.leave()

            # Εκτυπώνουμε τους successors και τους predecessors για κάθε
            # κόμβο
            elif choice == "3":
                print_successor_predecessor(chord_ring)
            # Εκτυπώνουμε τα δεδομένα όλων των κόμβων

```

```

elif choice == "4":
    print_node_data(chord_ring)

    # Αναζήτηση με lookup όπου ο χρήστης εισάγει σαν όρισμα το
    κλειδί και λαμβάνει τον κόμβο που αντιστοιχεί στο κλειδί αυτό με τα
    δεδομένα του
    elif choice == "5":
        key = int(input("Εισαγωγή του κλειδιού για αναζήτηση: "))
        result_node = chord_ring.Nodes_List[0].lookup(key)
        print(f"Ο κόμβος που αντιστοιχεί στο κλειδί {key} είναι ο
        κόμβος {result_node.node_id}")

        # Εκτύπωση των δεδομένων του κόμβου που βρέθηκε
        print(f"Δεδομένα του κόμβου {result_node.node_id}:
        {result_node.key_store}")

        # Εκτέλεση ενός query και αναζήτηση στον κόμβο με βάση το
        Πανεπιστήμιο και των αριθμό των βραβείων που εισήγαγε ο χρήστης
        elif choice == "6":
            node_id = int(input("Εισαγωγή του ID του κόμβου για την
            αναζήτηση: "))
            education = input("Εισαγωγή του Πανεπιστημίου: ")
            awards = int(input("Εισαγωγή του αριθμού των Βραβείων:
            "))

            for node in chord_ring.Nodes_List:
                if node.node_id == node_id:
                    # Εκτέλεση της επιθυμητής λειτουργίας για τον
                    επιλεγμένο κόμβο
                    scientists = []
                    for key, value in node.key_store.items():
                        # Έλεγχος αν το κλειδί είναι ίσο με την
                        επιλεγμένη εκπαίδευση ("education")
                        if key == education and int(value['awards'])
                        >= awards:
                            scientists.append(value['surname'])

                    if scientists:
                        print(f"Οι επιστήμονες που έκαναν βασικές
                        σπουδές στο {education} και έχουν αποσπάσει > {awards} βραβεία είναι:
                        {'', '.join(scientists)}")
                    else:
                        print("Δεν βρέθηκαν επιστήμονες που πληρούν

```

```
τα κριτήρια.")
    # Έξοδος από το σύστημα
elif choice == "7":
    print("Έξοδος από το πρόγραμμα. Αντίο!")
    break

else:
    # Μήνυμα για μη έγκυρη επιλογή από το menu των επιλογών
    print("Μη έγκυρη επιλογή. Παρακαλώ εισάγετε έναν αριθμό
μεταξύ του 1 και του 7.")
```